



BLM4531 Ağ Tabanlı Teknolojiler ve Uygulamaları
Final Raporu

Ad Soyad: Eda Nur ARSLAN

Öğrenci No: 22290210

Öğretim Görevlisi: Enver BAĞCI

GitHub Link: <https://github.com/edanurarslan/Office-Reservation-System-Web>

Video Link: <https://www.youtube.com/watch?v=SNmwYH-Rnqk>

İÇİNDEKİLER

- **BÖLÜM 1: Proje Özeti**
- **BÖLÜM 2: Kullanılan Teknolojiler**
- **BÖLÜM 3: Sistem Mimarisi**
- **BÖLÜM 4: Fonksiyonel Özellikler ve Modüller**
- **BÖLÜM 5: Görsel Tasarım ve Kullanıcı Deneyimi**
- **BÖLÜM 6: Proje Geliştirme Süreci ve İnovasyonlar**
- **BÖLÜM 7: Kalite Güvence ve Test Sonuçları**
- **BÖLÜM 8: Proje Gelişim Analizi**
- **BÖLÜM 9: Sonuç ve Değerlendirme**

1. PROJE ÖZETİ

Ofis Yönetim Sistemi, modern iş dünyasında ofis kaynaklarını optimize etmek amacıyla geliştirilmiş, kurumsal düzeyde bir yönetim platformudur. Sistem; çalışma alanlarının, toplantı odalarının ve çalışan etkileşimlerinin dijital bir ekosistem üzerinden uçtan uca yönetilmesini sağlar.

1.1. Projenin Temel Amacı ve Vizyonu

Projenin ana odağı, şirketlerin sahip olduğu fiziksel ofis kaynaklarını verimli bir şekilde yöneterek, çalışanların bu kaynaklara erişimini kolaylaştırmak ve operasyonel maliyetleri minimize etmektir. Sistem, manuel ilerleyen rezervasyon süreçlerini tamamen dijitalleştirerek insan hatasını azaltmayı ve veri odaklı bir yönetim anlayışını benimsemeyi hedefler.

1.2. Rol Bazlı Erişim ve Kişiselleştirme

Uygulama, karmaşık kurumsal hiyerarşileri destekleyen **Rol Tabanlı Erişim Kontrolü** mekanizması üzerine inşa edilmiştir:

- **Çalışanlar (Employee):** Kendi rezervasyonlarını kolayca yapabilir, QR kod teknolojisi ile hızlı check-in işlemlerini gerçekleştirebilir ve kişisel profil bilgilerini güncelleyebilir.
- **Yöneticiler (Manager):** Sorumlu oldukları ekiplerin rezervasyon süreçlerini denetleyebilir, onay iş akışlarını yönetebilir ve birim bazlı kullanım raporlarına erişebilir.
- **Sistem Yöneticileri (Admin):** Tüm ofis lokasyonlarını tanımlayabilir, sistem genelindeki iş kurallarını (maksimum rezervasyon süresi, çalışma saatleri vb.) belirleyebilir ve kapsamlı denetim günlüklerini inceleyebilir.

2. KULLANILAN TEKNOLOJİLER

Ofis Yönetim Sistemi, yüksek performans, ölçeklenebilirlik ve güvenlik standartları gözetilerek **Modern Full-Stack** yaklaşımlarıyla geliştirilmiştir. Projenin mimarisi, esnek ve güvenli bir arka yüz servisi ve kullanıcı deneyimini ön planda tutan bir ön yüzden oluşmaktadır.

2.1. Backend (Arka Yüz) Teknolojileri

- **ASP.NET Core 9 (C#):** Enterprise-grade, yüksek performanslı ve asenkron programlama desteği sunan RESTful API mimarisi.
- **JWT (JSON Web Token) & Refresh Token:** Kullanıcı kimlik doğrulama ve yetkilendirme süreçleri, güvenliği artırılmış token mekanizmaları ile yönetilmektedir.
- **Entity Framework Core:** Veritabanı etkileşimleri için modern bir **Object-Relational Mapping** yapısı kullanılarak kod seviyesinde veri yönetimi sağlanmıştır.

2.2. Frontend (Ön Yüz) Teknolojileri

- **React 19 & TypeScript:** Tip güvenliği yüksek, sürdürülebilir ve bileşen tabanlı modern bir kullanıcı arayüzü inşa edilmiştir.
- **Vite:** Geliştirme sürecinde yüksek hızda derleme ve optimize edilmiş çıktı performansı sağlamak amacıyla tercih edilmiştir.
- **Tailwind CSS & Material Design 3:** Utility-first CSS yaklaşımı ile modern ve Material Design 3 standartlarına uygun görsel tasarımlar uygulanmıştır.

2.3. Veri Yönetimi ve Altyapı

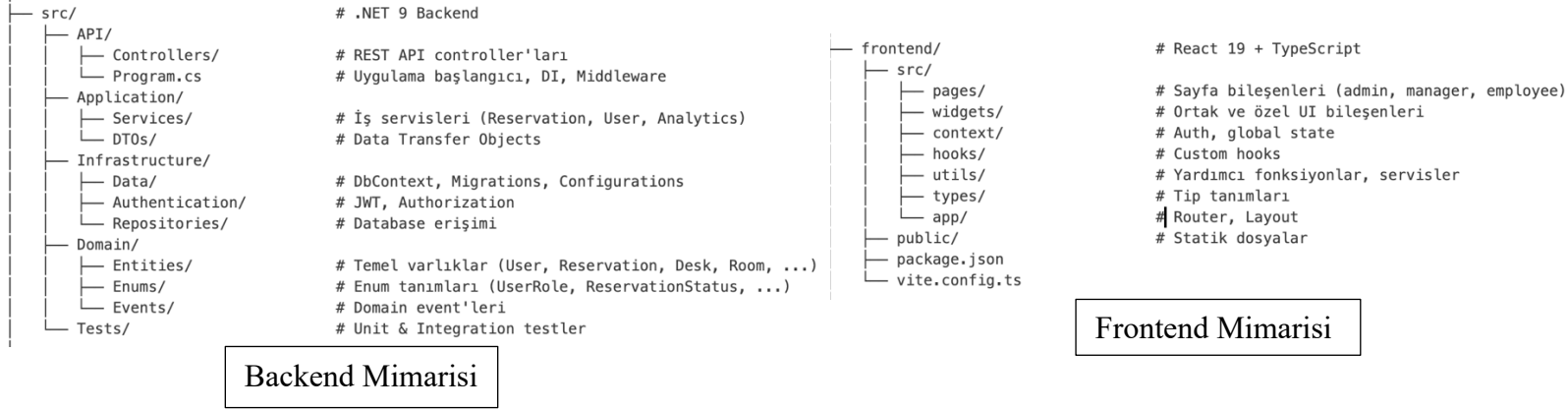
- **PostgreSQL:** Üretim ortamında yüksek veri tutarlılığı ve performans sunan ilişkisel veritabanı yönetim sistemi tercih edilmiştir.
- **Audit Logging:** Sistemdeki her kritik işlem denetim günlüğü tablolarında kayıt altına alınarak tam şeffaflık sağlanır.

2.4. Yardımcı Araçlar ve Mimari Desenler

- **Axios:** API isteklerinin merkezi bir noktadan yönetimi ve hata kontrolü için kullanılmıştır.
- **React Context API & Custom Hooks:** Global uygulama durumu yönetilmiş ve iş mantığı bileşenlerden ayrıştırılarak tekrar kullanılabilir hale getirilmiştir.
- **Role-Based Access Control (RBAC):** Admin, Manager ve Employee rolleri için kaynak bazlı yetki kontrolü uygulanmıştır.

3. SİSTEM MİMARİSİ

Ofis Yönetim Sistemi, modern web uygulama prensipleri doğrultusunda ayrıştırılmış iki ana katmandan oluşan bir mimari üzerine kurulmuştur. Bu yapı, uygulamanın bakımını kolaylaştırırken yüksek performans ve ölçeklenebilirlik sunar. Kod yapısı aşağıdaki gibi gösterilebilir:



3.1. Backend (Sunucu Katmanı)

Backend katmanı, Ofis Yönetim Sistemi'nin kalbi olarak işlev görmektedir. Bu katman, tüm iş mantığı, veri işleme, güvenlik kontrolleri ve veritabanı yönetimini gerçekleştirmektedir. Sistem mimarisinin en kritik parçası olan backend, .NET 9 teknolojisi kullanılarak geliştirilmiştir.

3.1.1 Genel Yapı ve Amaç

Backend katmanı, aşağıdaki temel görevleri yerine getirmektedir:

- Veri İşleme ve Yönetimi:** Tüm kullanıcı isteklerini alır, işler ve veritabanı ile iletişim kurar.
- İş Mantığı Uygulaması:** Rezervasyon çakışma kontrolleri, rol bazlı yetkilendirme, onay iş akışları gibi kompleks iş kurallarını uygular.
- Güvenlik:** JWT token tabanlı kimlik doğrulama, şifre hashing, veri şifreleme ve audit logging gibi güvenlik mekanizmalarını yönetir.
- API Sağlama:** Frontend ve diğer istemcilere RESTful API endpoint'leri sunarak yapılandırılmış veri alışverişini sağlar.

3.1.2 RESTful API Mimarisı

Backend, tam RESTful prensiplere uygun olarak tasarlanmış API endpoint'leri sunmaktadır. Bu endpoint'ler, HTTP metodları (GET, POST, PUT, DELETE) ve JSON veri formatı kullanarak istemci-sunucu haberleşmesini standartlaştırmaktadır.

Ana API Modülleri:

1. Authentication Module (/api/auth)

- o POST /api/auth/login - Kullanıcı giriş işlemi
- o POST /api/auth/register - Yeni kullanıcı kaydı
- o POST /api/auth/refresh-token - Token yenileme
- o GET /api/auth/me - Oturum açmış kullanıcı bilgisi
- o POST /api/auth/logout - Oturum kapatma

2. Reservations Module (/api/reservations)

- o GET /api/reservations - Tüm rezervasyonları listele
- o POST /api/reservations - Yeni rezervasyon oluştur
- o GET /api/reservations/{id} - Belirli bir rezervasyonun detaylarını görüntüle
- o PUT /api/reservations/{id} - Rezervasyonu güncelle
- o DELETE /api/reservations/{id} - Rezervasyonu iptal et
- o GET /api/reservations/availability/check - Masa/oda uygunluğu kontrolü
- o POST /api/reservations/{id}/approve - Yönetici onayı (Manager/Admin)
- o POST /api/reservations/{id}/reject - Yönetici reddi

3. Locations Module (/api/locations)

- o GET /api/locations - Tüm ofis konumlarını listele
- o POST /api/locations - Yeni konum oluştur (Admin)
- o GET /api/locations/{id} - Konumun detaylarını görüntüle
- o PUT /api/locations/{id} - Konumu güncelle (Admin)
- o DELETE /api/locations/{id} - Konumu sil (Admin)

4. Desks Module (/api/desks)

- o GET /api/desks - Tüm masaları listele
- o GET /api/desks/{id} - Masa detaylarını görüntüle
- o POST /api/desks - Yeni masa oluştur (Admin)

- o PUT /api/desks/{id} - Masa bilgisini güncelle (Admin)
 - o DELETE /api/desks/{id} - Masayı sil (Admin)
5. **Rooms Module (/api/rooms)**
- o GET /api/rooms - Tüm toplantı odalarını listele
 - o GET /api/rooms/{id} - Oda detaylarını görüntüle
 - o POST /api/rooms - Yeni oda oluştur (Admin)
 - o PUT /api/rooms/{id} - Oda bilgisini güncelle (Admin)
 - o DELETE /api/rooms/{id} - Odayı sil (Admin)
6. **Users Module (/api/users)**
- o GET /api/users - Kullanıcı listesi (Manager/Admin)
 - o GET /api/users/{id} - Kullanıcı detayları
 - o POST /api/users - Yeni kullanıcı oluştur (Admin)
 - o PUT /api/users/{id} - Kullanıcı bilgisini güncelle
 - o DELETE /api/users/{id} - Kullanıcıyı sil (Admin)
 - o POST /api/users/{id}/change-role - Kullanıcı rolünü değiştir (Admin)
7. **Check-in/Check-out Module (/api/checkins)**
- o POST /api/checkins - QR kodu ile check-in
 - o POST /api/checkins/{id}/checkout - Check-out işlemi
 - o GET /api/checkins - Check-in geçmişi
8. **Analytics Module (/api/analytics)**
- o GET /api/analytics/dashboard - Ana dashboard istatistikleri
 - o GET /api/analytics/reports - Detaylı raporlar
 - o GET /api/analytics/usage - Kullanım analizi (günlük/haftalık/aylık)
 - o GET /api/analytics/peak-hours - En yoğun saatler analizi
9. **Notifications Module (/api/notifications)**
- o GET /api/notifications - Kullanıcının bildirimlerini listele
 - o POST /api/notifications/{id}/read - Bildirimi oku olarak işaretle

- o DELETE /api/notifications/{id} - Bildirimi sil

10. Audit Logs Module (/api/logs)

- o GET /api/logs - Sistem audit loglarını listele (Admin)
- o GET /api/logs?userId={userId} - Belirli kullanıcının işlemlerini görüntüle

3.1.3 Kimlik Doğrulama ve Güvenlik JWT (JSON Web Token) Mekanizması

Sistem, JWT tabanlı stateless kimlik doğrulama kullanmaktadır:

1. Giriş Aşaması:

- o Kullanıcı email ve şifre ile giriş yapar
- o Backend, şifreyi bcrypt ile hashlenerek veritabanında saklanan şifre ile karşılaştırır
- o Eğer doğruysa, backend iki token üretir:
 - **Access Token:** 24 saat geçerli, API istekleri için kullanılır
 - **Refresh Token:** 30 gün geçerli, access token süresi dolduğunda yenileme için kullanılır

2. Token Yapısı:

```
{  
  "sub": "user-id",  
  "email": "user@example.com",  
  "role": "employee",  
  "exp": 1705353600,  
  "iat": 1705267200,  
  "jti": "unique-token-id"  
}
```

3. API İstek Akışı:

- o Frontend, her API isteğinde Authorization: Bearer {access_token} header'ını gönderir
- o Backend, token'ı doğrular ve geçerli ise isteği işler
- o Eğer token süresi dolmuşsa, 401 Unauthorized hatası döner
- o Frontend, refresh token kullanarak yeni access token alır

Rol Bazlı Erişim Kontrolü (RBAC)

Sistem, üç temel rol tanımlamaktadır:

1. Employee (Çalışan):

- o Kendi rezervasyonlarını oluşturabilir ve yönetebilir

- Diğer kullanıcıları görüntüleyemez
 - Dashboard'da yalnızca kendi istatistiklerini görebilir
 - QR koduyla check-in/out yapabilir
2. **Manager (Yönetici):**
- Takımın (Manager tarafından atanan) kullanıcıların rezervasyonlarını onaylayabilir/reddedebilir
 - Takımın istatistiklerini ve raporlarını görebilir
 - Takım üyelerinin bilgilerini güncelleyebilir
 - Sistem yapılandırmaları yapamaz
3. **Admin (Yönetici):**
- Tüm sistem işlemlerini yapabilir
 - Kullanıcı, konum, masa, oda yönetimi
 - İş kurallarını (min/max süre, işletme saatleri) belirleyebilir
 - Audit loglarını görüntüleyebilir
 - Sistem ayarlarını konfigüre edebilir

Güvenlik Mekanizmaları

1. **Şifre Güvenliği:**
- Bcrypt algoritması ile şifreler hashlenir (salt değeri ≥ 10)
 - Düz şifreler veritabanında saklanmaz
 - Şifre güncelleme sırasında eski şifre doğrulama
2. **Input Validation:**
- Tüm gelen veriler regex pattern'leri ile doğrulanır
 - Email, telefon, tarih formatları kontrol edilir
 - SQL injection ve XSS saldırılarına karşı koruma
3. **CORS Policy:**
- Geliştirme: http://localhost:5173 izin verilir
 - Production: Spesifik domain'ler için konfigüre edilir
 - Unauthorized domain'lerden gelen istekler reddedilir
4. **Rate Limiting:**
- Login endpoint'i: Dakikada 5 deneme (failsafe)
 - API istekleri: Dakikada maksimum 60 istek (per user)
 - Dosya upload: Maksimum 50 MB

3.1.4 İş Mantığı Katmanı (Application Services)

İş mantığı, Application katmanında servisler aracılığıyla uygulanmaktadır:

Reservation Service

```
// Rezervasyon oluřturma - akıřma kontrol
- Kullanıcı, masa/odayı, tarih ve saati seer
- Sistem, seili masa/odanın o saatte uygun olup olmadığını kontrol eder
- Eėer akıřma varsa, hata mesajı dner
- Eėer uygunsa, Status = "Pending" olarak kaydedilir
- Manager'a onay bildirimi gnderilir

// Onay iř akıřı
- Manager, Pending durumundaki rezervasyonları grr
- Onaylarsa, Status = "Confirmed" olarak gncellenir
- Kullanıcıya onay bildirimi gnderilir
- Redderse, Status = "Rejected" ve gereke kaydedilir

// İptal iřlemi
- Kullanıcı, bařlangı saatinden 2 saat ncesine kadar iptal edebilir
- İptal edilirse, masa/oda yeniden uygun hale gelir
- İptal kaydı audit log'a yazılır
```

User Service

```
// Kullanıcı oluřturma
- Admin, yeni kullanıcı oluřturur
- Sistem, otomatik gl řifre retir
- Email ile řifre gnderilir
- Kullanıcı, ilk giriř sırasında řifre deėiřtirebilir

// Rol atama
- Admin, kullanıcıya Employee, Manager veya Admin rol atayabilir
- Rol deėiřikliėi audit log'a kaydedilir
```

Analytics Service

```
// Dashboard istatistikleri
- Toplam kullanıcı, aktif kullanıcı sayısı
- Gnlk, haftalık, aylık rezervasyon sayıları
- Kullanım oranı (ka masa/oda kullanılıyor)

// Detaylı raporlar
- Kullanıcı bařına rezervasyon sayısı
- En ok kullanılan masa/odalar
- İptal oranları
- Peak hours (en yoėun saatler)
```

3.1.5 Veritabanı Katmanı (Data Access)

Entity Framework Core

- Database first yaklařımı ile PostgreSQL veritabanısı tasarlanmıřtır
- DbContext kullanılarak tm veritabanı iřlemleri ynetilir
- Migration'lar ile versiyon kontrol saėlanır

Repository Pattern

- Her entity için Repository sınıfı oluşturulmuştur
- Veritabanı sorgularının tekrarlanmasını önler
- Test edilebilirliği artırır

3.1.6 Exception Handling ve Error Management

Tüm hatalar merkezi olarak yönetilmektedir:

- 400 Bad Request: Geçersiz giriş verileri
- 401 Unauthorized: Kimlik doğrulama başarısız
- 403 Forbidden: Yetkilendirme başarısız
- 404 Not Found: Kaynak bulunamadı
- 409 Conflict: Çakışma (örn: aynı masaya iki kişi)
- 500 Internal Server Error: Sunucu hatası (logged)

3.1.7 Logging ve Monitoring

- Tüm API istekleri ve yanıtları loglanır
- Hatalar, stack trace'leri ile kaydedilir
- Kullanıcı işlemleri (login, rezervasyon, onay) Audit Log'a yazılır
- Performance metrikleri (response time, database query time) monitöre edilir

3.1.8 Scalability ve Performance

- Asynchronous işlemler (async/await) kullanılmaktadır
- Database query'leri optimize edilmiştir (indexing, eager loading)
- Caching mekanizmaları uygulanmıştır (Distributed Cache)
- Load balancing için prepared (stateless design)

3.2. Frontend (İstemci Katmanı)

Frontend katmanı, kullanıcıların Ofis Yönetim Sistemi ile etkileşim kurduğu en önemli bileşendir. Bu katman, modern React 19 teknolojisi, TypeScript güvenliği ve Tailwind CSS styling kullanılarak geliştirilmiştir. Sistemi kullanıcı dostu, hızlı ve responsive bir deneyim sunmaktadır.

3.2.1 Genel Yapı ve Amaç

Frontend katmanı, aşağıdaki temel görevleri yerine getirmektedir:

- **Kullanıcı Arayüzü (UI):** Profesyonel ve modern görünüm sunan Material Design 3 uyumlu bileşenler
- **Kullanıcı Deneyimi (UX):** Animasyonlar, geçişler ve interaktif öğelerle akıcı bir deneyim
- **İş Mantığı Yönetimi:** Frontend'de doğrulama, form işleme, state yönetimi
- **Backend İletişimi:** Axios aracılığıyla RESTful API ile asenkron haberleşme
- **Rol Bazlı Arayüz:** Kullanıcının rolüne göre dinamik sayfa ve menü yapılandırması

3.2.2 Teknoloji Stack'i

Core Teknolojiler

1. **React 19 (TypeScript)**
 - Component-based mimarisi ile modüler tasarım
 - Hooks (useState, useEffect, useContext, useReducer) ile state yönetimi
 - Strict Mode'de geliştirme, production'da optimize edilmiş kod
2. **Vite**
 - Lightning-fast build tool ve dev server
 - Hot Module Replacement (HMR) ile anlık kod güncellemeleri
 - Production build'de otomatik kod minification ve optimization
3. **TypeScript**
 - Tüm bileşenler type-safe yazılmıştır
 - Hata kakalamayı geliştirme aşamasında sağlar
 - IntelliSense ve auto-completion desteği
4. **Tailwind CSS**
 - Utility-first CSS framework ile hızlı styling
 - Material Design 3 prensiplerine uygun renk ve sizing
 - Responsive design (mobile-first) approach
5. **React Router v6**
 - Client-side routing ve sayfa geçişleri
 - Nested routes ve dynamic route parameters
 - Lazy loading ile performance optimization
6. **Lucide React**
 - 400+ modern ve tutarlı ikonlar

- SVG tabanlı ve skalabilir
- Tüm UI bileşenlerinde kullanılmıştır

7. Axios

- HTTP client library
- Request/response interceptor'ları
- Timeout, retry ve error handling mekanizmaları

3.2.4 Role Bazlı Arayüz Yönetimi

Frontend, kullanıcının rolüne göre dinamik olarak bileşenleri gösterir veya gizler:

Rol Bazlı Menü Yapılandırması

```
// Sidebar.tsx
const getMenuItems = () => {
  switch (user?.role) {
    case 'admin':
      return [
        { name: 'Genel Bakış', path: '/admin/overview', icon: LayoutDashboard },
        { name: 'Kullanıcılar', path: '/admin/users', icon: Users },
        { name: 'Konumlar', path: '/admin/locations', icon: Building },
        { name: 'Onaylar', path: '/admin/approval', icon: CheckSquare },
        { name: 'Loglar', path: '/admin/logs', icon: Activity },
        // ... diğer admin menü öğeleri
      ];
    case 'manager':
      return [
        { name: 'Dashboard', path: '/manager/dashboard', icon: LayoutDashboard },
        { name: 'Kullanıcılar', path: '/manager/users', icon: Users },
        { name: 'Raporlar', path: '/manager/reports', icon: BarChart3 },
        // ... diğer manager menü öğeleri
      ];
    case 'employee':
      return [
        { name: 'Dashboard', path: '/employee/dashboard', icon: LayoutDashboard },
        { name: 'Rezervasyonlar', path: '/employee/reservations', icon: Calendar },
        { name: 'Konumlar', path: '/employee/locations', icon: MapPin },
        // ... diğer employee menü öğeleri
      ];
  }
};
```

3.2.5 State Yönetimi (Context API)

Frontend, global state yönetimi için React Context API kullanmaktadır:

AuthContext - Kullanıcı ve Oturum Bilgisi

```
// useAuth hook'u tüm bileşenlerde kullanılır
const { user, isLoading, login, logout } = useAuth();

// İçinde saklanan veriler:
// - Oturum açmış kullanıcı bilgisi
// - JWT tokens (access + refresh)
// - Kullanıcı rolü ve yetkileri
// - Kimlik doğrulama durumu
```

Veri Akışı

1. Kullanıcı giriş yapar (LoginPage)
2. Axios, POST /api/auth/login çağrısı yapar
3. Backend JWT tokenları döner
4. AuthContext, tokens'ı localStorage'de saklar
5. Kullanıcı bilgisi global state'e yazılır
6. Tüm bileşenler useAuth() hook'u ile erişebilir
7. Protected routes, kullanıcı rolünü kontrol eder

API Çağrı Örnekleri

```
// Kullanıcı giriş
const login = async (email: string, password: string) => {
  const response = await axiosInstance.post('/auth/login', { email, password });
  return response.data; // { accessToken, refreshToken, user }
};

// Rezervasyon listesini al
const getReservations = async () => {
  const response = await axiosInstance.get('/reservations');
  return response.data; // [ Reservation[], ... ]
};

// Yeni rezervasyon oluştur
const createReservation = async (data: ReservationData) => {
  const response = await axiosInstance.post('/reservations', data);
  return response.data; // { id, status, ... }
};

// Rezervasyonu onayla (Manager)
const approveReservation = async (reservationId: string) => {
  const response = await axiosInstance.post(
    `/reservations/${reservationId}/approve`
  );
  return response.data;
};
```

3.3. İletişim Protokolü

Frontend ve Backend arasındaki haberleşme, modern web standartlarına uygun olarak tasarlanmıştır. Bu katman, güvenli, hızlı ve güvenilir bir veri alışverişi sağlamaktadır.

3.3.1 HTTP Protokolü ve REST Mimarisi

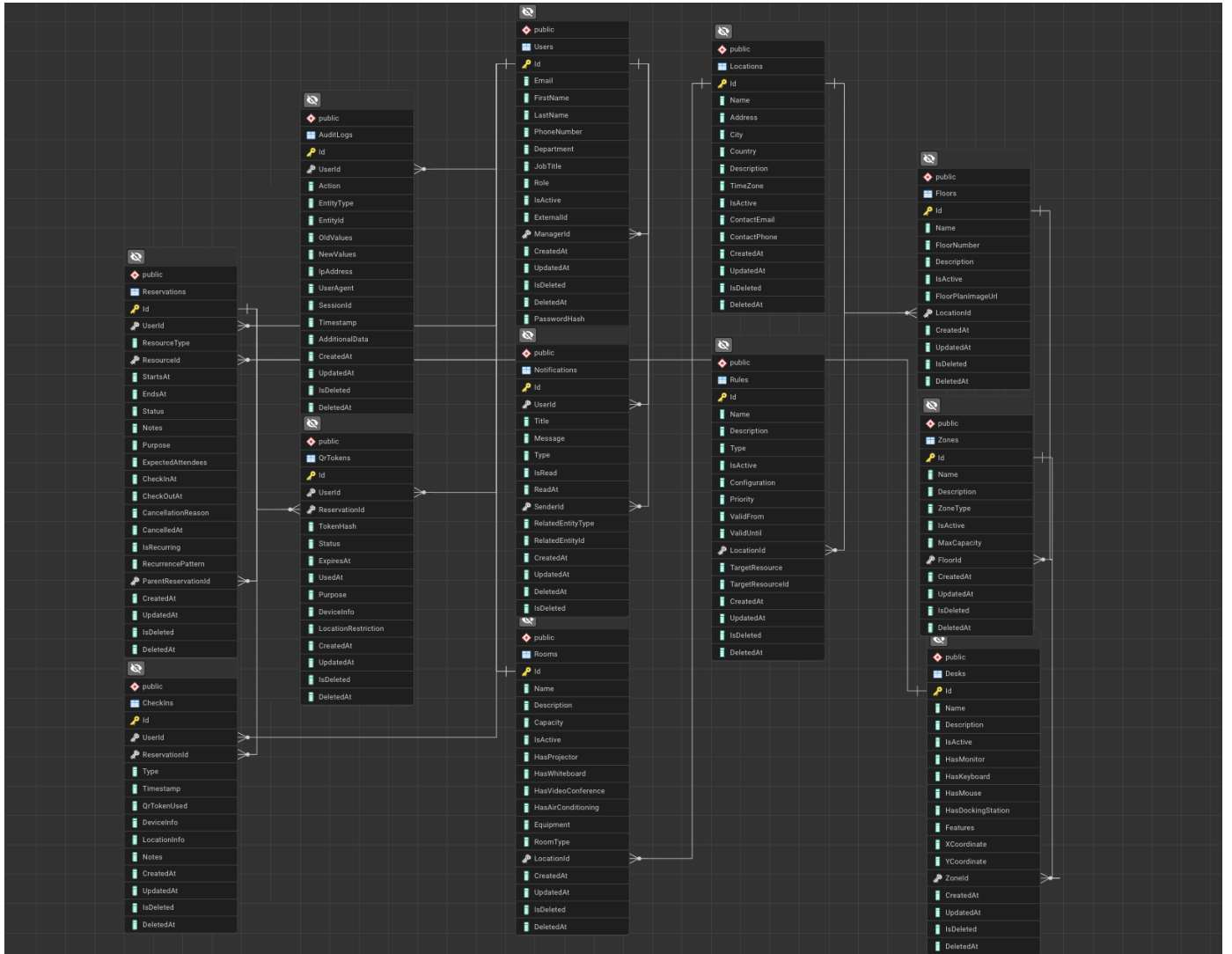
HTTP Protokolünün Kullanımı

Frontend ve Backend arasındaki tüm iletişim HTTP/HTTPS protokolü üzerinden gerçekleşmektedir:

- **Geliştirme Ortamı:** HTTP (localhost üzerinde)
- **Production Ortamı:** HTTPS (TLS 1.2+ ile şifreleme)

3.3.2 Diğer Yapılandırmalar

Geliştirme ortamında karşılaşılan tarayıcı güvenlik kısıtlamaları (CORS), Vite proxy yapılandırması ile optimize edilerek sorunsuz bir geliştirme akışı sağlanmıştır.



4. FONKSİYONEL ÖZELLİKLER VE MODÜLLER

Ofis Yönetim Sistemi, karmaşık ofis operasyonlarını basitleştirmek adına birbiriyle entegre çalışan üç ana modülden oluşmaktadır.

4.1. Kullanıcı ve Yetki Yönetimi

Sistem, kurumsal hiyerarşiyi destekleyen esnek bir kullanıcı yönetim paneline sahiptir:

- **Hiyerarşik Yapı:** Admin, Yönetici ve Çalışan rolleri için özelleştirilmiş dashboard ekranları ve işlem yetkileri tanımlanmıştır.
- **Profil Yönetimi:** Kullanıcıların avatar, kişisel bilgiler ve aktif/pasif durumlarının merkezi olarak yönetilmesi sağlanmaktadır.
- **Güvenlik:** Her işlem, rol bazlı yetkilendirme filtresinden geçirilerek veri güvenliği garanti altına alınmıştır.

4.2. Rezervasyon ve Onay İş Akışı

Ofis kaynaklarının verimli kullanımı için optimize edilmiş bir rezervasyon motoru sunulur:

- **Kaynak Rezervasyonu:** Çalışanlar, ofis içerisindeki masa ve toplantı odalarını interaktif bir arayüz üzerinden rezerve edebilirler.
- **Onay Mekanizması:** Rezervasyonlar, yönetici onayına sunulabilir veya belirlenen kurallar dahilinde otomatik olarak işlenebilir.
- **İstatistiksel Takip:** Günlük ve toplam rezervasyon verileri üzerinden kaynak kullanım verimliliği analiz edilebilir.

4.3. Bildirim Sistemleri ve Denetim Günlüğü

Sistem içi etkileşimi artırmak ve şeffaflık sağlamak amacıyla geliştirilmiştir:

- **Kişiselleştirilmiş Bildirimler:** Rezervasyon onayları, iptaller veya önemli hatırlatmalar kullanıcıya özel olarak iletilir.
- **Sistem Günlükleri:** Kritik tüm işlemler "Audit Log" yapısında kayıt altına alınarak adminler tarafından geriye dönük incelenebilir.

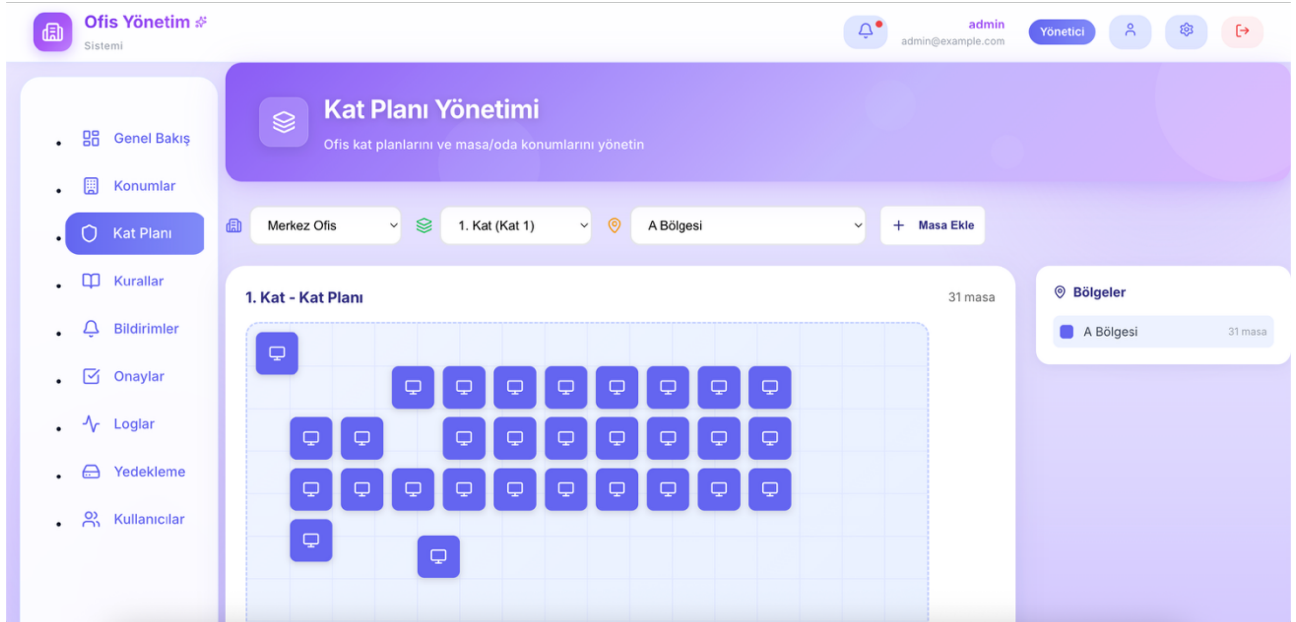
5. GÖRSEL TASARIM VE KULLANICI DENEYİMİ (UX)

Ofis Yönetim Sistemi'nin arayüzü, estetik değerleri yüksek tutarken kullanıcı odaklılıktan ödün vermeyen, modern web tasarım standartlarına uygun şekilde geliştirilmiştir.

5.1. Tasarım Felsefesi ve Görsel Kimlik

Uygulamanın genel görsel dili, profesyonel bir çalışma ortamını dijital dünyaya yansıtacak şekilde kurgulanmıştır:

- **Duyarlı Tasarım (Responsive):** **Glassmorphism** (buzlu cam) ile derinlik katılmış ve modernleştirilmiş tasarımının yanında esnek ızgara sistemleri sayesinde uygulama; masaüstü monitörlerden mobil cihazlara kadar tüm ekran boyutlarında kusursuz bir görüntüleme ve kullanım kolaylığı sunar.



Şekil 5.1.1. Genel Tasarım

5.2. Navigasyon ve Kullanıcı Akışı

Kullanıcının uygulama içerisinde kaybolmadan, hedeflediği işleme en hızlı şekilde ulaşması hedeflenmiştir:

- Ekranın sol tarafında sabitlenen menü(sidebar), kullanıcının sahip olduğu yetki seviyesine (Admin, Manager, Employee) göre çalışma anında dinamik olarak güncellenir.
- Üst bölümdeki gezinti çubuğu(navbar); kullanıcı profil özetine, sistem bildirimlerine ve kritik hızlı erişim butonlarına anlık erişim imkanı sağlar.



Şekil 5.2.1. Admin Dashboard

5.3. Temel Arayüz Bileşenleri

- **PageHeader:** Sayfa kimliğini güçlendiren ikonlu ve açıklayıcı başlık kutuları ile kullanıcıya bulunduğu bağlam net bir şekilde sunulur.
- **StatCard ve DashboardCard:** Kritik verilerin ve istatistiklerin kolayca taranmasını sağlayan, görsel hiyerarşisi güçlü modern veri kartları kullanılmıştır.
- **AnimatedCard:** Veri listeleme ve sunum ekranlarında kartların ekrana giriş animasyonları, arayüze dinamizm katar.
- **Form ve Tablo Mimarisi:** Veri giriş ve izleme süreçlerinin verimliliği için sade, okunaklı ve kullanıcıyı yormayan standart form ve tablo tasarımları uygulanmıştır.

The Login Screen for the 'Ofis Yönetim Sistemi' features a central white card on a purple gradient background. The card displays the system name, a login prompt, and input fields for 'E-posta Adresi' (admin@example.com) and 'Şifre'. A 'Giriş Yap' button is present, along with a note about demo account credentials. The footer includes a copyright notice for 2025.

Şekil 5.3.1. Giriş Ekranı

6. PROJE GELİŞTİRME SÜRECİ VE İNOVASYONLAR

Projenin geliştirme aşamasında, kullanıcı geri bildirimleri ve teknik gereksinimler doğrultusunda sistem performansını ve estetiğini artıran bir dizi kritik güncelleme yapılmıştır.

6.1. Görsel ve Arayüz Modernizasyonu

Uygulamanın kurumsal kimliğini güçlendirmek amacıyla tasarım dili baştan aşağı yenilenmiştir:

- **Tematik Güncelleme:** Tüm sayfa başlıkları ve veri kartları için mor ağırlıklı, derinlik hissi veren yeni bir kurumsal tema uygulanmıştır.
- **Efekt ve Derinlik:** Sidebar ve Navbar bileşenleri, **glassmorphism** ve gradyan efektleri kullanılarak modern web standartlarına taşınmıştır.
- **Dinamik Bileşenler:** Dashboard ve kullanıcı yönetimi sayfalarına, kullanıcı etkileşimini artıran animasyonlu kart geçişleri entegre edilmiştir.

6.2. Kullanıcı Deneyimi (UX) Optimizasyonu

Kullanıcıların sistemle olan etkileşimini kolaylaştırmak için aşağıdaki iyileştirmeler yapılmıştır:

- **Gelişmiş Yetki Kontrolü:** Rol bazlı menü yönetimi ve dinamik yetkilendirme sistemi, daha güvenli ve akıcı bir deneyim için optimize edilmiştir.
- **Geri Bildirim Mekanizmaları:** İşlem sonuçlarını bildiren hata ve başarı mesajları için modern, okunaklı uyarı kutuları (toasts/alerts) eklenmiştir.

6.3. Teknik Altyapı İyileştirmeleri

Sistemin sürdürülebilirliği ve performansı için kod seviyesinde düzenlemeler yapılmıştır:

- **API Standartlaştırma:** Tüm API uç noktaları (endpoints) /v1/ sürümleme mimarisine geçirilmiş ve proxy yapılandırmalarıyla stabil hale getirilmiştir.
- **Kod Mimarisi:** Projenin dosya ve bileşen yapısı, sürdürülebilirlik ilkelerine (SOLID) uygun olarak sadeleştirilmiş ve kod okunabilirliği artırılmıştır.

7. KALİTE GÜVENCE VE TEST SONUÇLARI

Uygulamanın kararlılığını ve güvenliğini doğrulamak amacıyla kapsamlı test süreçleri yürütülmüş, başarılı sonuçlar elde edilmiştir.

7.1. Fonksiyonel Doğrulama Testleri

Sistemin temel işlevleri farklı senaryolar altında test edilmiştir:

- **Çoklu Rol Testleri:** Admin, Yönetici ve Çalışan rollerinin sisteme giriş süreçleri ve yetki sınırları titizlikle kontrol edilmiştir.
- **Veri Yönetimi:** Kullanıcı ekleme, silme, düzenleme ve rol atama operasyonlarının veritabanı ile tam uyumlu çalıştığı doğrulanmıştır.
- **Rezervasyon Döngüsü:** Rezervasyon oluşturma, onaylama ve iptal süreçlerinin iş kurallarına uygunluğu ve çakışma kontrolleri başarıyla test edilmiştir.

7.2. Kullanıcı Deneyimi ve Performans Değerlendirmesi

Sistemin hızı ve erişilebilirliği ölçümlenmiştir:

- **Akışkanlık:** Sayfa geçişleri ve animasyonların tüm tarayıcılarda hızlı ve takılmasız çalıştığı gözlemlenmiştir.
- **Hızlı Derleme:** Vite mimarisi sayesinde uygulama yükleme süreleri optimize edilmiş, geliştirme ortamında yüksek performans sağlanmıştır.

8. PROJE GELİŞİM ANALİZİ

Projenin başlangıçtaki teknik şartname hedefleri ile nihai uygulama aşamasında ulaşılan sonuçlar karşılaştırmalı olarak ele alınmıştır.

8.1. Başlangıç Planlaması ve Beklentiler

Proje ilk aşamada, şirketlerin ofis kaynaklarını verimli yönetmesini sağlayan teknik bir şartname olarak kurgulanmıştır. Temel hedefler şunlardı:

- **Çok Katmanlı Mimari:** .NET 9 tabanlı stateless REST API'ler ve React frontend ile sağlam bir mimari temel oluşturulması.
- **Kritik Fonksiyonlar:** Masa/oda rezervasyonu, QR kod tabanlı check-in/out sistemi ve yönetici onay mekanizmalarının kurulması.
- **Veri Güvenliği:** PostgreSQL 14 kullanımı, JWT tabanlı kimlik doğrulama ve Role-Based Access Control yapısının tesisi.

8.2. Uygulama Süreci ve Nihai Çıktılar

Geliştirme süreci sonunda, başlangıçtaki vizyon sadece teknik olarak tamamlanmakla kalmamış, kullanıcı deneyimi açısından daha ileri bir noktaya taşınmıştır:

- **Teknolojik Adaptasyon:** Planlanan .NET ve React stack'ine ek olarak, arayüzde modern **Glassmorphism** ve **Gradient** tasarımları uygulanarak kurumsal estetik artırılmıştır.
- **Fonksiyonel Genişleme:** Temel rezervasyon işlemlerine ek olarak, analitik dashboard'lar, doluluk ısı haritaları (Heatmap) ve kapsamlı denetim günlükleri sisteme başarıyla entegre edilmiştir.
- **Operasyonel Verimlilik:** Başlangıçta hedeflenen "Noshow" yönetimi ve otomatik iptal mekanizmaları, arka plan işleri ile tam otomatize edilerek sistemin kendi kendini yönetme kabiliyeti artırılmıştır.

9. SONUÇ VE DEĞERLENDİRME

Ofis Yönetim Sistemi, başlangıçta belirlenen teknik şartname ve MVP hedeflerine tam uyumlu şekilde tamamlanmıştır. Proje süreci boyunca hedeflenen tüm teknik ve fonksiyonel gereksinimler, modern web standartlarıyla harmanlanarak hayata geçirilmiştir.

9.1. Proje Kazanımları ve Çıktılar

- **Tam Fonksiyonellik:** Rezervasyon, kullanıcı yönetimi, QR doğrulama ve konum bazlı operasyonların tamamı kararlı bir yapıda sunulmuştur.
- **Üst Düzey Kullanıcı Deneyimi:** Teknik karmaşıklığı düşük, adaptasyon süreci hızlı ve estetik açıdan zengin bir kullanım ortamı oluşturulmuştur.
- **Sürdürülebilir Mimari:** Temiz kod (clean code) prensipleri ve modüler yapı sayesinde, sistem gelecekteki **AI destekli ekip koordinasyon önerileri** (V3 vizyonu) gibi güncellemelere hazır hale getirilmiştir.

Sistem, gerçek ofis ortamlarına kolayca uyarlanabilir yapısıyla dijital dönüşüm adına güçlü bir temel sunmaktadır.